

Benjamin Garcia

(626) 246-4778 | bentgarcia05@gmail.com | [GitHub](#) | [LinkedIn](#) | [Portfolio](#)

EDUCATION

University of California, Los Angeles

Bachelor of Science, Computer Science

Los Angeles, CA

Expected Graduation: Dec. 2027

Relevant Coursework: Data Structures and Algorithms, Operating Systems, Computer Architecture, Programming Languages

TECHNICAL SKILLS

Languages: TypeScript, Python, C++, Java, SQL, JavaScript

Systems & Backend: Node.js, FastAPI, Flask, **asyncio**, Temporal, Redis, PostgreSQL, SQLite, MySQL

Infrastructure: Docker, Linux/Unix, Nginx, Prometheus, Grafana, GitHub Actions, AWS Lambda, Git

Frontend & AI: React, Next.js, GraphQL, WebSockets, Playwright, OpenAI API

EXPERIENCE

Undergraduate Researcher

Dec. 2025 – Present

UCLA Human-Computer Interaction Research Lab

Los Angeles, CA

- Building a **human-in-the-loop AI research platform** that retrieves scientific literature, constructs grounded research perspectives, and coordinates multi-agent deliberation with researcher-gated updates to versioned artifacts.
- Evaluated competing retrieval architectures across **5 research domains and 20 uncached runs**, then implemented a hybrid two-round Semantic Scholar pipeline combining semantic embeddings, density-based clustering, diversity-aware selection, and deterministic fallbacks.
- Engineered independently versioned document state with **revision-checked persistence**, backed by **173 pytest and 40 Playwright tests** spanning retrieval, interaction flows, and failure recovery.

Production Engineering Fellow

Jun. 2026 – Sep. 2026

Meta

Remote

- Deployed and operated a containerized **Flask/MySQL service on CentOS** behind Nginx with production environment configuration, DuckDNS routing, and defensive handling across deployment and runtime failures.
- Automated **CI/CD deployments** with **unittest**-based test gates, structured logging, and failure-aware deployment checks.
- Instrumented host and container health with **automated service checks, Prometheus metrics, Grafana dashboards, and alerting** for production observability.

Software Engineer Intern

Jun. 2026 – Aug. 2026

Lindy

San Francisco, CA

- Built Lindy's **multi-tenant AI-agent guardrails and approvals platform** with a policy store, arg-hashed approval tokens, fail-closed runtime enforcement, and Slack/web flows; retained **5,104 production decisions across 674 workspaces**.
- Took over a stalled Integrations unification effort and drove personal and team integrations to **100% GA**, shipping a feature-flagged migration foundation that preserved existing workflows while consolidating the platform and net-deleting **4,521 lines**.
- Led engineering for Lindy's existing-user migration to Teammate, shipping Slack authorization, durable memory import, a five-step migration flow, and E2E launch gates; go-live reached **78 production completions and 50 activations**.
- Diagnosed a silent **21-minute production Slack hydration stall** caused by KEDA scale-down and five code defects, then migrated orchestration from BullMQ to Temporal in one day; the following 14-day window recorded **409 runs with one failure**.

PROJECTS

Annie | TypeScript, Node.js, Fastify, SQLite, OAuth 2.0/OIDC, Docker, Railway

Aug. 2026 – Present

- Profiled production agent traces and redesigned the reply critical path with deferred memory, concurrent provider reads, and batched Gmail fetches, reducing **live two-read latency by 17.2%** across paired benchmarks.
- Engineered a SQLite-backed agent runtime with **lease-token fencing, per-chat FIFO, bounded backpressure, and durable write intents** that never automatically replay provider mutations after unknown acceptance.
- Built repairable tracing and credential-free replay across model, tool, write, egress, and memory events, using the telemetry to isolate production latency and failure paths.

PolicyC | TypeScript, Python, asyncio, SQLite, OpenAI API, Zod, FastAPI

Jul. 2026 – Present

- Built a source-preserving policy compiler, cutting mean input from **16.4K to 3.2K tokens** and billed API cost by **34%** in an eight-case, 64-response evaluation.
- Built a resumable Python **asyncio** runtime with atomic persistence and cost controls, achieving **3.60× effective throughput** across three held-out experiments.
- Evaluated **1,680 GPT-5 mini executions across 280 held-out cases**, measuring **76–86% preservation** of full-prompt critical-policy passes through strategy-blind grading.